About

Superteam Brazil is building the **ultimate learning platform for Solana-native developers** — an open-source, interactive education hub that takes builders from zero to deploying production-ready dApps.

Think "Codecademy meets Cyfrin Updraft" for Solana: **gamified** progression, **interactive** coding challenges, **on-chain** credentials, and a **community**-driven learning experience built for crypto natives.

We're looking for talented developers and cracked agents to build this platform from the ground up. The winning submission will become the foundation for Solana developer education across Latin America and beyond.

Overview

Build a production-ready learning management system (LMS) for Solana development, research, and power users.

The platform should:

- Deliver interactive, project-based courses with integrated code editing
- Track learner progress with gamification (XP, streaks, achievements)
- Issue on-chain credentials for course completion
- Support multiple languages (Portuguese, Spanish, English)
- Integrate analytics for user behavior insights
- Be fully open-source and forkable by other communities

Rewards

- □ **1st Place** — $3,500 USDG
- □ **2nd Place** — $1,000 USDG
- □ **3rd Place** — $500 USDG

Total Prize Pool: $4,800 USDC

On-Chain Program

The platform's gamification and credential logic lives on-chain via an Anchor program. The full spec and code live at github.com/solanabr/superteam-academy — your delivery should be a **PR to this repo** (inside it's according folder following monorepo structure):

/root

.claude/ (skills & agents)

docs/

onchain-academy/ (program)

-> app (front end client)

-> backend (back end client)

Here's what you need to know:

- **XP is a soulbound fungible token** (Token-2022, NonTransferable). A learner's token balance = their XP. Levels are derived: Level = floor(sqrt(xp / 100)).
- **Credentials are Metaplex Core NFTs**, soulbound via PermanentFreezeDelegate. One NFT per learning track that upgrades in place as the learner progresses — no wallet clutter. Attributes like track, level, courses completed, and total XP are stored on-chain.
- **Courses are on-chain PDAs** that spawn Enrollment PDAs per learner. Lesson progress is tracked via a 256-bit bitmap (up to 256 lessons per course).
- **Enrollments are closeable** after completion to reclaim rent. Proof is preserved via the credential NFT and transaction history.
- **Streaks are a frontend-only feature** — daily activity tracking, streak history visualization, and milestone rewards are not tracked on-chain and should be implemented in the frontend (local storage, database, or CMS).
- **Achievements use a bitmap** (256 possible) AchievementType defines the badge (name, metadata URI, supply cap, XP reward) and AchievementReceipt records each award to a learner, each backed by a soulbound Metaplex Core NFT..
- **Leaderboard is off-chain** — derived by indexing XP token balances (Helius DAS API or custom indexer).

For the full program specification, see docs/SPEC.md. For frontend integration patterns (PDA derivation, instruction usage, events, error codes), see docs/INTEGRATION.md.

What to Implement vs. Stub

Fully implement on Devnet:

- Wallet authentication (multi-wallet adapter)
- XP balance display from Token-2022 token accounts
- Credential (Metaplex Core NFT) display and verification
- Leaderboard by indexing XP balances
- Course enrollment — the learner signs the enroll transaction directly (no backend needed)

Stub with clean abstractions (we connect the on-chain program later)**:**

- Lesson completion flow (backend-signed transactions)
- Course finalization and credential issuanc
- Achievement claiming
- Streak tracking (frontend-only)

Create clean service interfaces so we can swap local storage for on-chain calls. For example, a **LearningProgressService** should expose methods like: get progress for a user/course, complete a lesson, get XP balance, get streak data, get leaderboard entries (by weekly/monthly/all-time timeframe), and get credentials for a wallet. See docs/INTEGRATION.md for the exact account structures, instruction parameters, and event signatures your service layer should map to.

Tech Stack

Choose **one** frontend framework:

- **React** + Next.js 14+ (App Router)
- **Vue 3** + Nuxt 3
- **Svelte** + SvelteKit

Required technologies:

- **TypeScript** — strict mode, no any types
- **Tailwind CSS** — custom theme with design tokens

- **Components** — shadcn/ui, Radix, or Headless UI (accessible, composable primitives)
- **Headless CMS** — Sanity, Strapi, Contentful, or similar
- **Auth** — Solana Wallet Adapter (multi-wallet) + Google sign-in. GitHub sign-in as bonus.
- **Analytics** — GA4 + heatmaps (Hotjar/PostHog/Clarity) + Sentry error monitoring
- **i18n** — PT-BR, ES, EN from day one. All UI strings externalized, language switcher in header/settings. Course content can remain in original language.
- **Deployment** — Vercel/Netlify with preview deployments

Code Editor Integration

Implement using **one** of: embedded Solana Playground (iframe), Monaco Editor, or CodeMirror 6.

Must support: Rust/TypeScript/JSON syntax highlighting, basic autocompletion, error display, and pass/fail feedback for challenges.

Account Linking

Users should be able to:

- Sign up with wallet OR Google OR GitHub
- Link additional auth methods later
- Use any linked method to sign in
- Wallet linking is required to finalize courses and receive credentials

Scope of Work

1. Landing Page (/)

Hero with value proposition and primary CTAs (Sign Up, Explore Courses). Learning path previews with progression indicators. Social proof (testimonials, partner logos, completion stats). Platform feature highlights. Footer with links, social, newsletter signup.

2. Course Catalog (/courses)

Filterable course grid by difficulty, topic, and duration. Curated learning paths (e.g., "Solana Fundamentals", "DeFi Developer"). Course cards with thumbnail, title, description, difficulty, duration, progress %. Full-text search.

3. Course Detail (/courses/[slug])

Course header with title, description, instructor, duration, difficulty. Expandable module/lesson list with completion status. Progress bar and XP to earn. Enrollment CTA. Reviews section (can be static for MVP).

4. Lesson View (/courses/[slug]/lessons/[id])

Split layout: content (left) + code editor (right), resizable. Markdown rendering with syntax highlighting. Previous/Next navigation and module overview. Lesson completion tracking with auto-save. Expandable hints and solution toggle.

5. Code Challenge Interface

Challenge prompt with clear objectives and expected output. Visible test cases with pass/fail indicators. Pre-populated starter code, editable. Run button with loading state and output display. Real-time error messages and success celebration. Mark complete and award XP.

6. User Dashboard (/dashboard)

Current courses with completion % and next lesson. XP balance, level progress bar, and rank. Current streak with calendar visualization. Recent achievements and badges. Recommended next courses. Recent activity feed.

7. User Profile (/profile, /profile/[username])

Profile header with avatar, name, bio, social links, join date. Skill radar chart (Rust, Anchor, Frontend, Security, etc.). Achievement badge showcase. On-chain credential display — evolving cNFTs with track, level, and verification links. Completed courses list. Public/private visibility toggle.

8. Leaderboard (/leaderboard)

Global rankings by XP. Weekly/monthly/all-time filters, filterable by course. User cards with rank, avatar, name, XP, level, streak. Current user highlighted.

9. Settings (/settings)

Profile editing (name, bio, avatar, social links). Account management (email, connected wallets, Google/GitHub). Preferences (language, theme, notifications). Privacy (profile visibility, data export).

10. Certificate/Credential View (/certificates/[id])

Visual certificate with course name, date, and recipient. On-chain verification link (Solana Explorer). Social sharing buttons and downloadable image. NFT details: mint address, metadata, ownership proof.

Gamification System

XP & Leveling

XP comes from on-chain soulbound tokens. Display the balance and derive level: Level = floor(sqrt(totalXP / 100)).

Rewards are tracked on-chain through interaction with the program at github.com/solanabr/superteam-academy. For the stubbed implementation, track XP locally — on-chain minting will be connected later.

XP rewards (configurable per course):

- Complete lesson — 10–50 XP (based on difficulty)
- Complete challenge — 25–100 XP
- Complete course — 500–2,000 XP
- Daily streak bonus — 10 XP
- First completion of the day — 25 XP

Streaks

Track consecutive days with activity. Visual calendar showing streak history. Streak freeze (bonus feature). Milestone rewards at 7, 30, and 100 days. Streaks are a frontend-managed feature — implement them using local storage or your database/CMS.

Achievements/Badges

- **Progress** — "First Steps", "Course Completer", "Speed Runner"
- **Streaks** — "Week Warrior", "Monthly Master", "Consistency King"
- **Skills** — "Rust Rookie", "Anchor Expert", "Full Stack Solana"
- **Community** — "Helper", "First Comment", "Top Contributor"

- **Special** — "Early Adopter", "Bug Hunter", "Perfect Score"

On-chain, achievements are managed through AchievementType and AchievementReceipt PDAs. Each achievement award mints a soulbound Metaplex Core NFT to the recipient. AchievementTypes support configurable supply caps and XP rewards.

CMS Integration

Courses contain modules, modules contain lessons. Each lesson is either **content** (reading/video) or **challenge** (interactive coding).

The CMS should support: visual content editor with markdown and code blocks, media management, draft/publish workflow, and course metadata (difficulty, duration, XP, track association).

We'll provide a **mock course** with sample content for testing.

Performance

Lighthouse targets: Performance 90+, Accessibility 95+, Best Practices 95+, SEO 90+.

Core Web Vitals: LCP < 2.5s, FID < 100ms, CLS < 0.1.

Implement image optimization, code splitting, lazy loading, static generation where possible, and bundle size optimization.

Deliverables

Required

- **Pull Request** to github.com/solanabr/superteam-academy with the full frontend implementation
- **Production application** — All 10 core pages functional, wallet auth, gamification system, code editor integration, i18n (PT-BR, ES, EN), light/dark themes, responsive, Lighthouse targets met
- **Analytics** — GA4 with custom events, heatmap solution, Sentry error monitoring
- **CMS** — Configured with content schema and sample course imported
- **Deployment** — Live demo on Vercel/Netlify with preview deployments
- **Documentation:**

    - **README.md** — Overview, tech stack, local dev setup, env vars, deployment
    - **ARCHITECTURE.md** — System architecture, component structure, data flow, service interfaces (including on-chain integration points)
    - **CMS_GUIDE.md** — How to create/edit courses, content schema, publishing workflow
    - **CUSTOMIZATION.md** — Theme customization, adding languages, extending gamification
- **Demo video** (3–5 min) — Feature walkthrough, architecture overview, key decisions
- **Twitter post** — Share submission, tag @SuperteamBR

Bonus

- Admin dashboard for course management and user analytics
- E2E tests (Playwright or Cypress) covering critical flows
- Community/forum section with discussion threads and Q&A
- Onboarding flow with skill assessment quiz
- PWA support (installable, offline-capable)
- Advanced gamification (daily challenges, seasonal events)
- CMS Course creator dashboard
- Actual integration with devnet program

Evaluation Criteria

- **Code Quality & Architecture (25%)** — Clean, typed, well-structured, maintainable. Clean service abstractions for future on-chain integration.
- **Feature Completeness (25%)** — All required features working correctly.
- **UI/UX Design (20%)** — Polished, intuitive, developer-focused. Dark mode primary.
- **Performance (15%)** — Lighthouse scores, load times, responsiveness.
- **Documentation (10%)** — Clear, comprehensive, useful for future developers.
- **Bonus Features (5%)** — Additional features beyond requirements.

Submission Requirements

Submit through **Superteam Earn** with:

1. **PR Link** — Pull request to github.com/solanabr/superteam-academy
2. **Live Demo URL** — Deployed application with easy signup flow and Devnet wallet for testing
3. **Demo Video** (3–5 min) — Feature walkthrough and architecture overview
4. **Twitter Post** — Share submission, tag @SuperteamBR

Timeline

- **Submission Deadline:** 14 days from listing
- **Winner Announcement:** Within 10 days after deadline
- **Payment:** Within 15 days after winner announcement

Resources

On-Chain Program: github.com/solanabr/superteam-academy

Frameworks: Next.js · Nuxt 3 · SvelteKit

UI: shadcn/ui · Radix UI · Headless UI

Solana: Wallet Adapter · Metaplex Docs · Solana Cookbook

CMS: Sanity · Strapi · Contentful

Code Editors: Monaco Editor · CodeMirror 6 · Solana Playground

Analytics: GA4 · PostHog · Hotjar · Clarity · Sentry

Design Inspiration: Cyfrin Updraft · Codecademy · Scrimba · LeetCode · Duolingo

Brand Assets: Solana Brand Kit · Superteam Brazil Brand Kit

Terms & Conditions

- All submissions must be original work
- Code must be open-source (MIT license)
- Winning submission will be used and extended by Superteam Brazil/LATAM
- Non-winning submissions remain property of the developer
- By submitting, you agree to potential follow-up collaboration
- Multiple submissions from the same person/agent/team are not allowed

- AI-generated code is allowed, but must be reviewed, tested, and production-quality
- Judges' decisions are final

FAQ

Q: Do I need to implement the full on-chain program? A: No. The on-chain program is in the repo already. Build the frontend with clean service interfaces. Credential display (reading NFTs from Devnet) should work — lesson completion and enrollment can be stubbed.

Q: Can I use additional libraries? A: Yes, as long as they don't conflict with required technologies. Document any additions.

Q: Do I need to create course content? A: No, we provide mock content. You build the platform and CMS structure.

Q: Can I use Supabase for user data? A: Yes, for MVP. Design clean abstractions so we can swap to on-chain later.

Q: What if I can't complete all features in 21 days? A: Prioritize core features over bonuses. A polished subset beats a buggy complete set.

Q: Can a team submit? A: Yes. Prize is per submission, not per person.

Q: Can I submit in Portuguese/Spanish? A: Yes! Non-English LATAM content is welcome.

Questions?

- **Discord:** discord.gg/superteambrasil
- **Twitter:** @SuperteamBR

Skills Needed: Frontend Development (React/Vue/Svelte), TypeScript, Tailwind CSS, Solana/Web3, CMS Integration

Eligibility: Global (+ agents)